CHAPTER 7

Monitoring Flask Applications

Up until now, we have covered developing, testing, and deploying our flask application. In this chapter, we'll discuss about some add-ons to manage and support your Flask application and steps forward from here.

Application Monitoring

Even after performing various kinds of tests on our application, in real world there are always some exceptional scenarios and bottlenecks which we are unaware of while development, and they shoot up as bugs and errors in production, as people start using the application. That's when we need application monitoring which monitors the behavior of your application in production including downtime check, endpoint errors, crashes, exceptions, and performance-related issues. Monitoring is crucial for applications, since raw log files are hard to interpret, and they get bulky overtime for developers to make sense of. Log files can detect functional errors most of the time, but it doesn't tell much about the performance-related issues which is also crucial to your application since the business is dependent upon the application being able to serve its customers in a timely fashion. Thus proactive application monitoring is critical to deal with stability, performance, and errors in your application.

© Kunal Relan 2019
K. Relan, *Building REST APIs with Flask*, https://doi.org/10.1007/978-1-4842-5022-8_7

All major Cloud service providers support features for monitoring virtual machines. We deploy our application on, for CPU utilization, memory utilization and network on the operating system level. However, application monitoring usually encompasses the following things:

1. Application errors and warnings
2. Application performance of every transaction
3. Database and third-party integration querying performance
4. Basic server monitoring and metrics
5. Application log data

There are a lot of great application monitoring tools in the market, and here in this chapter, we'll cover integrating a few of them as they all come with different sets of features, payment options, and so on.

Here is a list of few open-source monitoring projects:

1. Sentry
2. Sensu
3. Service canary
4. Flask monitoring dashboard

There are a couple of monitoring services available in the market as well like New Relic, Sentry.io, Scout, and so on which takes the burden of deploying the monitoring software away but comes at a price. We'll examine the types of application monitoring tools.

Sentry

Sentry is an open source application monitoring system developed in Python but is available for all major platforms. Sentry also has a Cloud-hosted service which has different subscription models ranging from a free version for developers to business versions which costs around $80 per month. In this we'll be checking out the free version and integrate it with ours which is pretty straightforward.

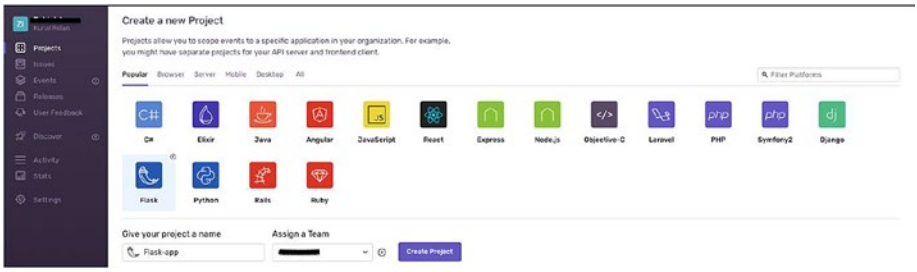

Figure 7-1. *Create new project*

To begin with, signup for sentry on `https://sentry.io/signup/`, and once you have successfully signed up, login to your dashboard. Once done, click add project button and select Flask as the framework, input the name, and submit on create project.

Once done, it will take us to the next page which has integration details, so just install the sentry SDK using PIP with the given command and copy paste the integration code in your main.py.

```
(venv)$pip install sentry-sdk[flask]
```

Next add the following code before app = Flask(__name__).

```
sentry_sdk.init(
    dsn="<your_dsn_here>"
    integrations=[FlaskIntegration()]
)
```

Once done, you can deploy the application and sentry will take care of the rest. Let's test it out by hitting a 404 endpoint making an event in sentry dashboard.

In your browser, request any endpoint which doesn't exist, and sentry SDK will fire off an event from the application like in the following screenshot.

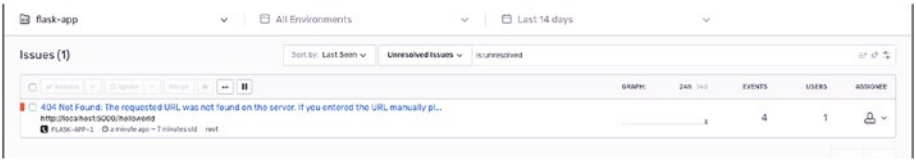

Figure 7-2. *Sentry issues listing*

Clicking on the issue will give you insights into the issue and details to help you resolve the issue like in the following screenshot.

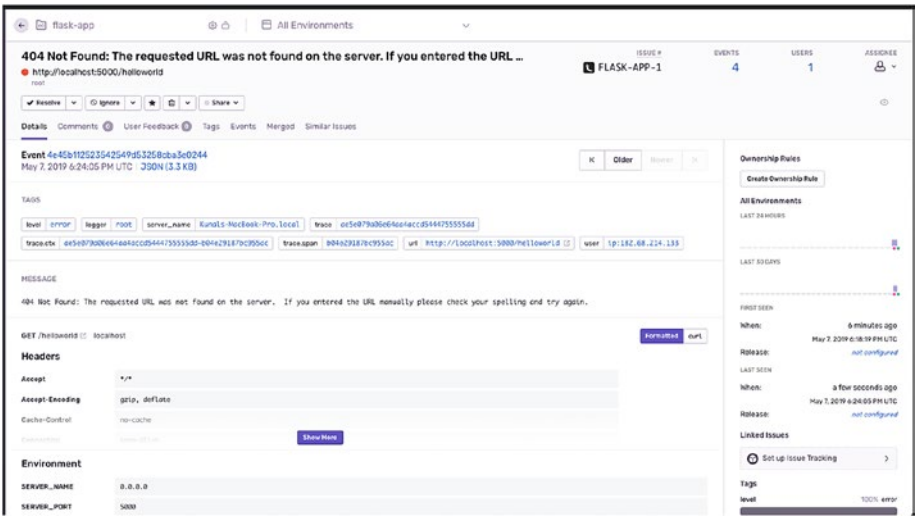

Figure 7-3. *Sentry issue details*

Sentry has a lot more features for you to explore and helo your application stable and error free.

Flask Monitoring Dashboard

Flask Monitoring Dashboard is an extension for Flask applications. It outrightly monitors the application performance and utilization, profiles the requests, and can also be configured to run specific jobs to manage your application. It is open source and free, so let's get started on it.

Install Flask Monitoring Dashboard using PIP with the following code.

```
(venv)$pip install flask_monitoringdashboard
```

Next in your main.py file, import the extension using the following code, where we have other library imports.

```
import flask_monitoringdashboard as dashboard
```

Now right below where we have initiated our app object, add the following code.

```
dashboard.bind(app)
```

And that's pretty much it; now restart your application and visit http://<host>:<port>/dashboard, and it will open the login page. The default credentials are admin and admin which you should change right after.

Figure 7-4. *Flask Monitoring Dashboard*

Once you login, you'll be redirected to the dashboard which will have an overview of your endpoints, and you can click each of them to get a deeper insight and can also set the monitoring level for each one of them as per your preferences.

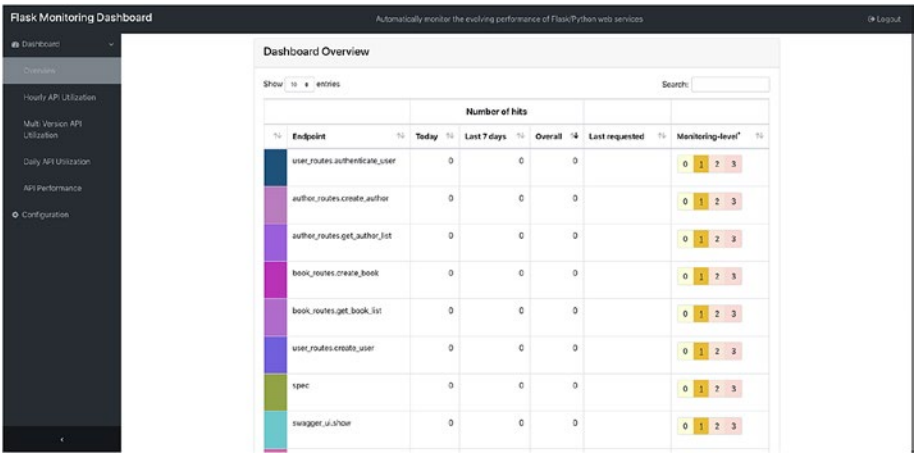

Figure 7-5. *Flask Monitoring Dashboard overview*

Clicking upon endpoints will redirect you to the insights page for each endpoint where you can churn out graph data for each requests and get more in-depth information about the endpoint usage.

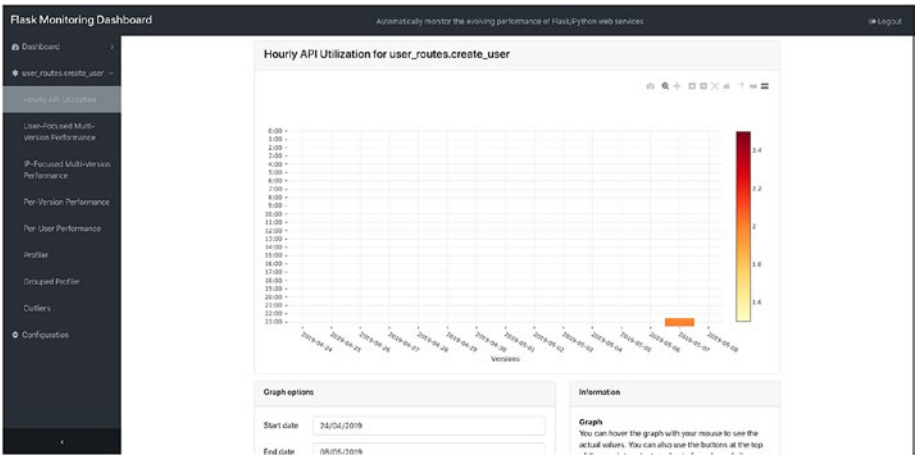

Figure 7-6. *API utilization insights*

New Relic

New Relic is one of the most reliable and competitive application monitoring tools in the market; they provide a comprehensive database with real-time monitoring services. New Relic is a paid subscription-based service, but they have a 14-day trial which we'll use to check it out. So go ahead and sign up for the same on `https://newrelic.com/signup`

Once you sign up, you'll be asked to choose the platform; go ahead and select Python as the platform.

Figure 7-7. *New Relic setup*

In your application directory, go ahead and install the New Relic agent using the following command.

```
(venv)$ pip install newrelic
```

Next we need to generate the configuration using our New Relic key, so click Reveal license key button on the screen which will display your private key.

Now execute the following command with your key.

```
(venv)$ newrelic-admin generate-config <your-key-goes-here>
newrelic.ini
```

So now our application should be configured; next use the following command to run your application and test the configuration.

```
(venv)$ NEW_RELIC_CONFIG_FILE=newrelic.ini newrelic-admin run-
program python run.py
```

Once you do, click Listen for my application button like in the following screenshot, which will start listening to requests from your application to configure your dashboard, and once it successfully receives the data, you'll see a button to redirect to your New Relic dashboard.

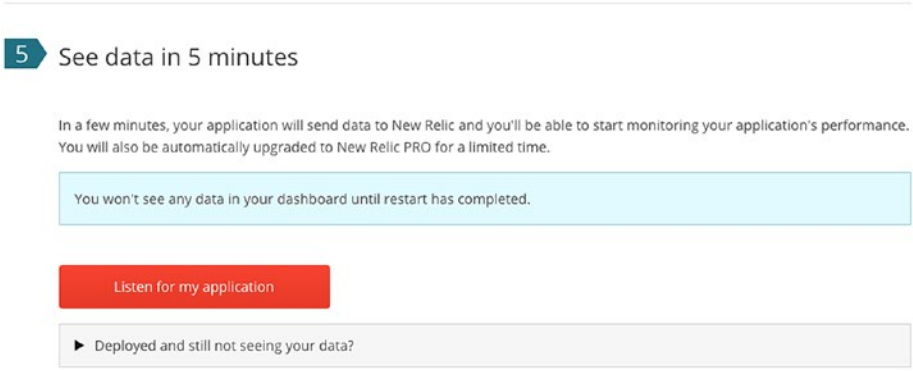

Figure 7-8. *New Relic Listen for application*

Now you'll see the application dashboard which lists your applications; click Python Application since we only have one application here.

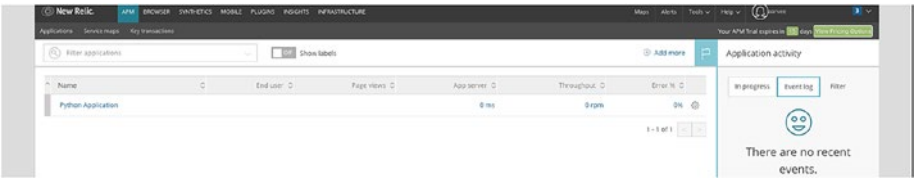

Figure 7-9. *New Relic Application dashboard*

After clicking, you'll be redirected to your application dashboard which looks quite comprehensive but provides a lot of insights about the application state. Refer to the following screenshot as an example.

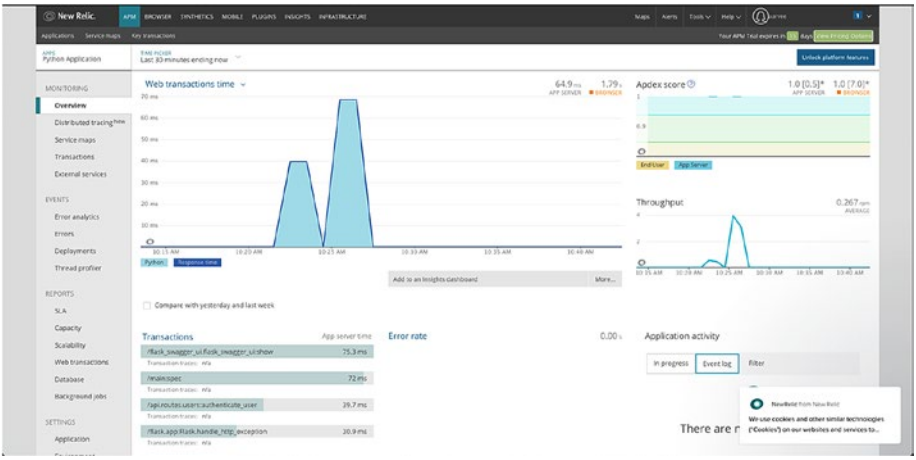

Figure 7-10. *Python Application dashboard*

As you can see, we can check out the latest transactions and check out the transaction time graph, and then we have throughput graph and apdex score which scores the application on the basis of a set value of 0.5 seconds response time. Under events, you can check out error analytics and errors which provide an in-depth details of errors raised in the system.

You can learn more about New Relic in their official documentation at https://docs.newrelic.com/docs/apm/new-relic-apm/guides.

Bonus Services

So we have covered all the topics to get you started with developing REST applications using Flask. However, this is just a beginner's guide, and there are a lot more things to cover for real business use cases including integrating third-party services like search, caching, pub-sub, real-time communication, and so on. This book covers the basics of Flask REST API development which can be used as a base to build your REST applications. In this module we'll cover the basics of some additional libraries and tools that could add value to your application.

Full Text Search with Flask

Flask is compatible with a couple of libraries that provide integration with full text search applications like Elasticsearch.

Pyelasticsearch

Pyelasticsearch is a clean library to integrate Elasticsearch which is a very popular and powerful search engine in your flask application. However, Elasticsearch provides REST endpoints to connect with the search engine, but pyelasticsearch makes it easier to communicate with the endpoints. You can learn more about pyelasticsearch at https://pyelasticsearch.readthedocs.io and check this link out to understand more about elasticsearch www.elastic.co/guide/en/elasticsearch/reference/current/getting-started.html

Flask-WhooshAlchemy

Whoosh is a fast, search engine library built in Python and is highly flexible and supports complex data searching based on free-form or structured text. Flask-WhooshAlchemy is a Flask extension to integrate the Whoosh search engine library with SQLAlchemy in Flask. It's pretty straightforward to integrate and get your full text search up without any third-party application unlike pyelasticsearch. You can learn more about it at https://pythonhosted.org/Flask-WhooshAlchemy/

Email

We used Flask mail for verifying user emails. Here is a list of other libraries you can use for efficient email integration in your application.

Flask-SendGrid

Flask-SendGrid is a Flask extension to simplify sending email using sendgrid which is a renowned email service; you can sign up for SendGrid at https://sendgrid.com/; they provide a free subscription which includes 40,000 emails for first 30 days and 100/day forever. You can check out Flask-SendGrid at https://github.com/frankv/flask-sendgrid which makes sending email a piece of cake.

AWS SNS Using Boto3

Boto3 is an AWS SDK for Python, and you can use AWS' SNS (Simple Notification Service) to send email and text message using Boto. Here is a guide to sending email using Python with Boto https://docs.aws.amazon.com/ses/latest/DeveloperGuide/send-using-sdk-python.html. You'll need an active AWS account for this and Boto installed from https://boto3.readthedocs.io/en/latest/guide/quickstart.html#installation.